SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	**Question**	**Bloom's Level**	**Marks**
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	**5**
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	**5**
3	Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	**5**
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	**5**
5	Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	**5**
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	**5**
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	**5**

8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	**5**
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	**5**
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	**5**

Code of Conduct:

*I __**S.Jaya Shree(192524146)**_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: __S.Jaya Shree___

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

QUESTION 1

For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

AIM

To identify the functional dependencies present in the given STUDENT_COURSE relation and determine the candidate key using the attribute closure method.

GIVEN RELATION

STUDENT_COURSE (SID, SName, CID, CName, Instructor, Grade)

Where:

Attributes and Meaning

Attribute	Meaning
SID	Student ID
SName	Student Name
CID	Course ID
CName	Course Name
Instructor	Course Instructor
Grade	Grade obtained by the student in the course

FUNCTIONAL DEPENDENCIES

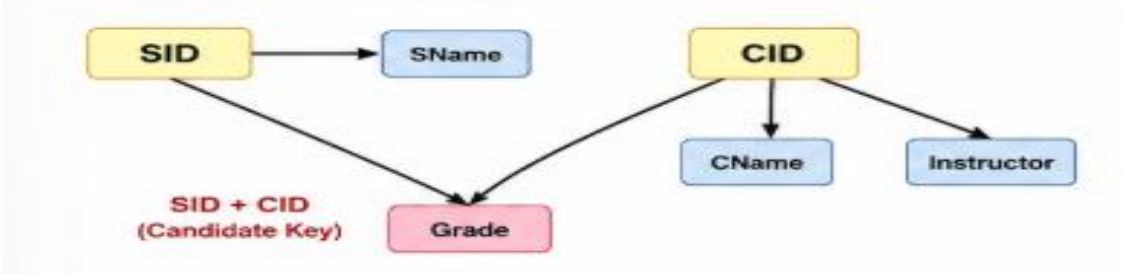
Based on the meaning of the attributes, the following functional dependencies exist:

FD1:	SID → SName	A student ID uniquely determines the student name.
FD2:	CID → CName	A course ID uniquely determines the course name.
FD3:	CID → Instructor	A course ID uniquely determines the instructor.
FD4:	(SID, CID) → Grade	The grade is determined by the combination of student ID and course ID.

F = { SID → SName, CID → CName, CID → Instructor, (SID, CID) → Grade }

FUNCTIONAL DEPENDENCY DIAGRAM

Figure 1: Functional Dependency Diagram of STUDENT_COURSE



The diagram shows that SID determines SName, CID determines CName and Instructor, and the combination of SID and CID determines Grade.

DETERMINATION OF CANDIDATE KEY

To determine the candidate key, the attribute closure of {SID, CID} is calculated.

SID	SName	CID	CName	Instructor	Grade
S101	Jaya	C01	DBMS	Kumar	A
S101	Jaya	C02	OS	Ravi	B+
S102	Priya	C01	DBMS	Kumar	A+
S103	Arun	C03	CN	Meena	B

- SID = S101 always gives SName = Jaya.
- CID = C01 always gives CName = DBMS and Instructor = Kumar.
- (SID, CID) together determine the Grade.

➤ **Step 1: Consider $(SID, CID)^+$**

Step	Current Set	FD Used	New Attributes	Resulting Set
0	{ SID, CID }	–	–	{ SID, CID }
1	{ SID, CID }	SID → SName	SName	{ SID, CID, SName }
2	{ SID, CID, SName }	CID → CName	CName	{ SID, CID, SName, CName }
3	{ SID, CID, SName, CName }	CID → Instructor	Instructor	{ SID, CID, SName, CName, Instructor }
4	{ SID, CID, SName, CName, Instructor }	(SID, CID) → Grade	Grade	{ SID, CID, SName, CName, Instructor, Grade }

$(SID, CID)^+$ = { SID, SName, CID, CName, Instructor, Grade } = All attributes

Therefore, { SID, CID } is a superkey.

MINIMALITY CHECK

To verify whether {SID, CID} is a candidate key, each attribute is checked individually.

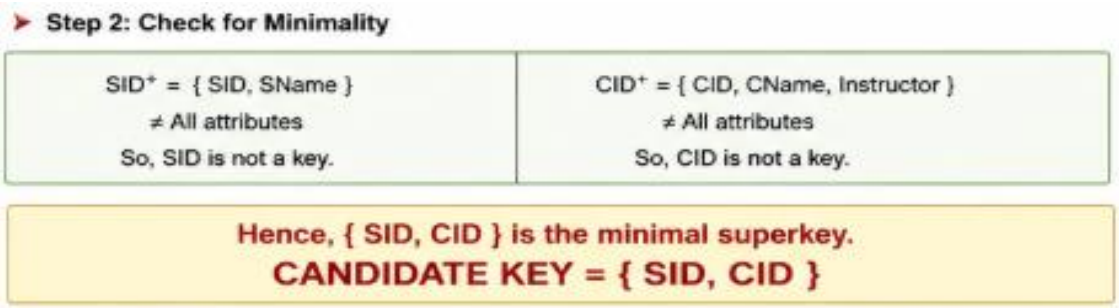
SID+ = {SID, SName}

Since SID alone does not determine all the attributes, SID is not a candidate key.

CID+ = {CID, CName, Instructor}

Since CID alone does not determine all the attributes, CID is not a candidate key.

Therefore, both SID and CID are required to uniquely identify a record.



CANDIDATE KEY

Candidate Key = {SID, CID}

The candidate key is a composite key consisting of SID and CID.

RESULT

The functional dependencies of the STUDENT_COURSE relation were successfully identified. The attribute closure of {SID, CID} contains all the attributes of the relation.

Hence, the candidate key of the given relation is:

{SID, CID}

CONCLUSION

Thus, the functional dependencies were identified and the candidate key was determined using the attribute closure method. The combination of SID and CID uniquely identifies each student-course enrollment.

QUESTION 2

Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

AIM

To demonstrate the conversion of an unnormalized relation into **First Normal Form (1NF)** by eliminating repeating groups and ensuring that all attributes contain atomic values.

THEORY

First Normal Form (1NF) is the first stage of database normalization. A relation is said to be in 1NF when:

1. Each attribute contains only **atomic (single) values**.
2. There are no repeating groups or multivalued attributes.
3. Each row represents a unique record.
4. Each column contains values of the same type.

UNNORMALIZED RELATION

Consider the following unnormalized relation:

STUDENT_COURSE

SID	SName	Course_ID	Course_Name	Grade
S101	Jaya	C01, C02	DBMS, OS	A, B+
S102	Priya	C01, C03	DBMS, CN	A+, B
S103	Arun	C02	OS	A

In this relation, a student can have multiple course IDs, course names and grades stored in the same row.

Therefore, the relation is **not in 1NF** because these attributes contain multiple values.

PROBLEM IN THE UNNORMALIZED RELATION

The following attributes contain repeating/multivalued data:

- Course_ID → C01, C02
- Course_Name → DBMS, OS
- Grade → A, B+

These values are not atomic.

Therefore:

STUDENT_COURSE is not in 1NF.

CONVERSION FROM UNF TO 1NF

To convert the relation into 1NF, each repeating value is separated into an individual row.

Step 1: Identify the repeating group

The repeating group is:

Course_ID, Course_Name and Grade
For example:
S101 → C01, C02
S101 → DBMS, OS
S101 → A, B+
These values must be separated.

Step 2: Create separate rows
The repeated values are converted into individual records.
1NF RELATION
STUDENT_COURSE_1NF

SID	SName	Course_ID	Course_Name	Grade
S101	Jaya	C01	DBMS	A
S101	Jaya	C02	OS	B+
S102	Priya	C01	DBMS	A+
S102	Priya	C03	CN	B
S103	Arun	C02	OS	A

Now every cell contains exactly **one value**.

1NF CONVERSION DIAGRAM

Figure 2: Conversion from Unnormalized Form to 1NF
UNNORMALIZED RELATION

JUSTIFICATION FOR 1NF
The resulting relation satisfies the requirements of 1NF because:

1. Every cell contains a **single atomic value**.
2. There are no repeating groups.
3. Each course enrollment is represented by a separate row.
4. The relation can be uniquely identified using the combination of **SID and Course_ID**.
5. Each column contains values of a consistent type.

Therefore, the relation has been successfully converted from **UNF to 1NF**.

PRIMARY KEY

The combination of:

(SID, Course_ID)

can be used as the **composite primary key**, because a student can enroll in multiple courses and each course can have multiple students.

RESULT

The unnormalized STUDENT_COURSE relation was successfully converted into **First Normal Form (1NF)** by eliminating repeating groups and converting all multivalued attributes into atomic values.

SID	SName	Course_ID	Course_Name	Grade
S101	Jaya	C01	DBMS	A
S101	Jaya	C02	OS	B+
S102	Priya	C01	DBMS	A+
S102	Priya	C03	CN	B
S103	Arun	C02	OS	A

CONCLUSION

Thus, the conversion from UNF to 1NF ensures that every attribute contains a single atomic value and removes repeating groups. The resulting 1NF relation provides a structured foundation for further normalization into **2NF and 3NF**.

QUESTION 3

Given a relation **R(A, B, C, D, E)** with functional dependencies **A → BC, BC → D, D → E**. Analyze and decompose it to **2NF**, eliminating partial dependencies.

AIM

To analyze the given relation and determine whether it satisfies **Second Normal Form (2NF)** by identifying and eliminating partial functional dependencies.

GIVEN RELATION

Functional Dependencies

1. A → BC
2. BC → D
3. D → E

Therefore,

Relation	R(A, B, C, D, E)
Functional Dependencies	1. A → BC 2. BC → D 3. D → E
F = { A → BC, BC → D, D → E }	

1. FIND THE CANDIDATE KEY

To determine the candidate key, calculate the closure of **A**.

Calculate closure of A^+

Step	Functional Dependency Used	Attributes in A^+
0	–	{ A }
1	A → BC	{ A, B, C }
2	BC → D	{ A, B, C, D }
3	D → E	{ A, B, C, D, E }

Since A^+ contains all attributes of R,
Candidate Key = { A }

2. CHECK FOR PARTIAL DEPENDENCIES

A relation is in **2NF** when:

1. It is already in **1NF**.
2. There is no partial dependency of a non-prime attribute on a proper subset of a composite candidate key.

Here, the candidate key is:

The candidate key contains **only one attribute**. Therefore, there cannot be a proper subset of the candidate key.

Hence, **partial dependency does not exist**.

Important Observation

Although:

- A → BC
- BC → D
- D → E

exist, these are **not partial dependencies**.

A partial dependency occurs only when a non-key attribute depends on **part of a**

composite key.

Since the key is only **A**, no partial dependency is possible.

3. 2NF ANALYSIS

Condition	Observation
Relation is in 1NF	Yes
Candidate key	A
Candidate key is composite?	No
Partial dependency exists?	No
Relation is in 2NF?	**Yes**

4. DECOMPOSITION

Since there are **no partial dependencies**, no decomposition is required for achieving 2NF.

Therefore, the 2NF relation remains:

2NF Decomposition Diagram

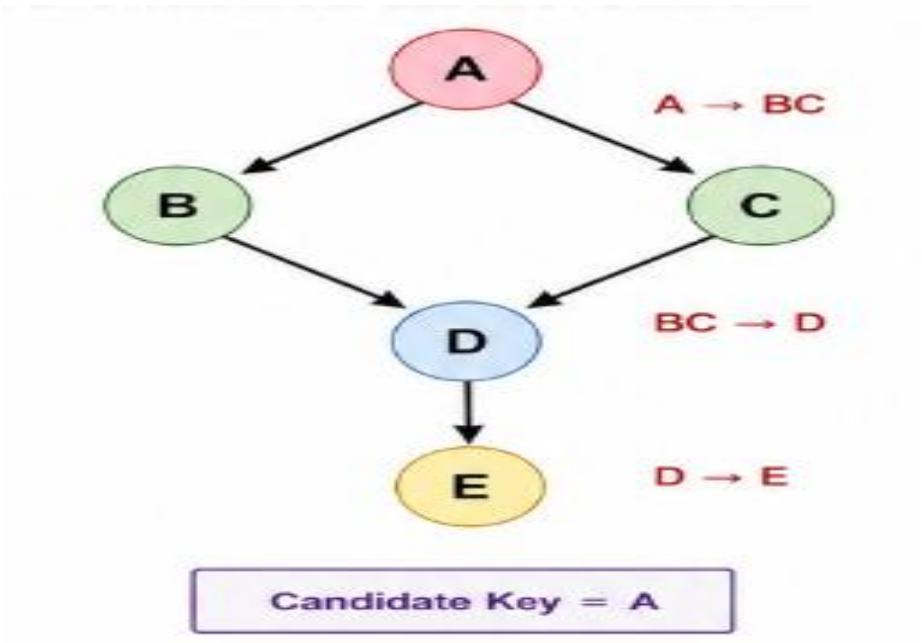
Figure 3: 2NF Analysis and Decomposition

Final 2NF Relation			
R(A, B, C, D, E)			
Relation	Attributes	Candidate Key	2NF Achieved
R	A, B, C, D, E	A	Yes

R(A, B, C, D, E)
↓
Candidate Key = A
↓
No partial dependency
↓
No decomposition required
↓
2NF: R(A, B, C, D, E)

5. FUNCTIONAL DEPENDENCY DIAGRAM

Figure 4: Functional Dependency Diagram



Candidate Key = A

This diagram shows the dependency chain:

A → BC → D → E

6. JUSTIFICATION

The relation is already in **2NF** because:

1. The candidate key is **A**.
2. A is a single-attribute candidate key.
3. Partial dependency can occur only with a composite candidate key.

4. Since there is no composite candidate key, there is no partial dependency.
5. Therefore, no decomposition is required to achieve 2NF.

RESULT

The candidate key of the relation R(A, B, C, D, E) is A. Since the candidate key consists of a single attribute, there are no partial dependencies.

Therefore:

No decomposition is required for 2NF.

CONCLUSION

Thus, the given relation satisfies **Second Normal Form (2NF)** because it has a single-attribute candidate key and contains no partial dependencies. The original relation can therefore be retained without decomposition for 2NF.

QUESTION 4

Apply 3NF decomposition on the relation **EMPLOYEE(EmpID, EmpName, DeptID, DeptName, Manager, ManagerID)** with transitive dependencies.

AIM

To analyze the given EMPLOYEE relation, identify transitive dependencies, and decompose the relation into **Third Normal Form (3NF)**.

GIVEN RELATION

EMPLOYEE (Emp ID, Emp Name, Dept ID, Dept Name, Manager, Manager ID)

Where:

Attribute	Description
EmpID	Employee ID
EmpName	Employee Name
DeptID	Department ID
DeptName	Department Name
Manager	Manager Name
ManagerID	Manager ID

FUNCTIONAL DEPENDENCIES

The functional dependencies are:

1. **EmpID → EmpName**
2. **EmpID → DeptID**
3. **DeptID → DeptName**
4. **DeptID → ManagerID**
5. **ManagerID → Manager**

Therefore:

F = { EmpID → EmpName, EmpID → DeptID, DeptID → DeptName, DeptID → ManagerID, ManagerID → Manager }

1. IDENTIFY THE CANDIDATE KEY

Step	FD Used	Result ($EmpID^+$)
0	–	{ EmpID }
1	EmpID → EmpName	{ EmpID, EmpName }
2	EmpID → DeptID	{ EmpID, EmpName, DeptID }
3	DeptID → DeptName	{ EmpID, EmpName, DeptID, DeptName }
4	DeptID → ManagerID	{ EmpID, EmpName, DeptID, DeptName, ManagerID }
5	ManagerID → Manager	{ EmpID, EmpName, DeptID, DeptName, ManagerID, Manager }

$EmpID^+$ contains all attributes.
Candidate Key = { EmpID }

2. IDENTIFY TRANSITIVE DEPENDENCIES

A transitive dependency occurs when a non-key attribute depends on another non-key attribute through the primary key.

- EmpID → DeptID and DeptID → DeptName
  ⇒ EmpID → DeptID → DeptName (DeptName is transitively dependent on EmpID)
- EmpID → DeptID and DeptID → ManagerID and ManagerID → Manager
  ⇒ EmpID → DeptID → ManagerID → Manager (Manager is transitively dependent on EmpID)

To achieve 3NF, remove transitive dependencies.

3. 3NF DECOMPOSITION

To remove the transitive dependencies, decompose the original relation into the following relations.

RELATION 1: EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

Attribute	Description
EmpID	Employee ID
EmpName	Employee Name
DeptID	Department ID

Primary Key: EmpID
FDs: EmpID → EmpName, DeptID

RELATION 2: DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

Attribute	Description
DeptID	Department ID
DeptName	Department Name
ManagerID	Manager ID

Primary Key: DeptID
FDs: DeptID → DeptName, ManagerID

RELATION 3: MANAGER

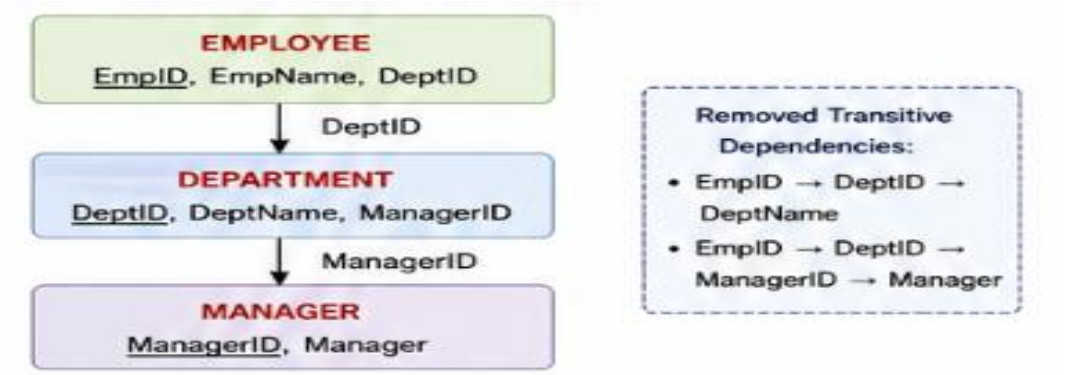
MANAGER(ManagerID, Manager)

Attribute	Description
ManagerID	Manager ID
Manager	Manager Name

Primary Key: ManagerID
FDs: ManagerID → Manager

4. 3NF DECOMPOSITION DIAGRAM

Figure 4: 3NF Decomposition of EMPLOYEE



The decomposition removes the transitive dependency:

EmpID → DeptID → DeptName

And **EmpID → DeptID → ManagerID → Manager**

5. VERIFY 3NF

EMPLOYEE

EmpID → EmpName, DeptID

EmpID is the candidate key, so the relation satisfies 3NF.

DEPARTMENT
DeptID → DeptName, ManagerID
DeptID is the candidate key, so the relation satisfies 3NF.

MANAGER
ManagerID → Manager
ManagerID is the candidate key, so the relation satisfies 3NF.
Therefore, all three relations satisfy **3NF**.

Relation	FDs in Relation	Key	3NF Status
EMPLOYEE	EmpID → EmpName, DeptID	EmpID	✔ Yes
DEPARTMENT	DeptID → DeptName, ManagerID	DeptID	✔ Yes
MANAGER	ManagerID → Manager	ManagerID	✔ Yes
All relations are in 3NF.			

6. FINAL 3NF RELATIONS

Relation	Attributes	Primary Key
EMPLOYEE	EmpID, EmpName, DeptID	EmpID
DEPARTMENT	DeptID, DeptName, ManagerID	DeptID
MANAGER	ManagerID, Manager	ManagerID

RESULT
The transitive dependencies in the EMPLOYEE relation were identified and eliminated successfully. The original relation was decomposed into three relations:

EMPLOYEE(EmpID, EmpName, DeptID)
DEPARTMENT(DeptID, DeptName, ManagerID)
MANAGER(ManagerID, Manager)

The resulting relations satisfy Third Normal Form (3NF).

CONCLUSION
Thus, the EMPLOYEE relation was successfully decomposed into 3NF by removing transitive dependencies. The decomposition reduces data redundancy and prevents update, insertion, and deletion anomalies while maintaining the relationships between employees, departments, and manager

QUESTION 5

Verify whether the relation **R(A, B, C, D)** with functional dependencies **A → C, C → D, D → A** is in BCNF. If not, decompose it into BCNF.

AIM

To verify whether the given relation satisfies **Boyce-Codd Normal Form (BCNF)** and, if necessary, decompose the relation into BCNF relations.

GIVEN RELATION

R(A, B, C, D)

Functional Dependencies

1. **A → C**
2. **C → D**
3. **D → A**

Therefore:

F = { A → C, C → D, D → A }

1. FIND THE CANDIDATE KEYS

Since none of the functional dependencies determines **B**, every candidate key must contain **B**.

Attribute Set	Closure
$A^+ = \{A, C, D\}$	(B is missing)
$C^+ = \{A, C, D\}$	(B is missing)
$D^+ = \{A, C, D\}$	(B is missing)
$(AB)^+ = \{A, B, C, D\}$	→ Superkey
$(BC)^+ = \{A, B, C, D\}$	→ Superkey
$(BD)^+ = \{A, B, C, D\}$	→ Superkey

Candidate Keys = {AB}, {BC}, {BD}

2. CHECK BCNF

A relation is in **BCNF** if, for every non-trivial functional dependency:

X → Y

X must be a **superkey**.

FD1: A → C

A^+ = {A, C, D}

Since B is missing, A is **not a superkey**.

Therefore:

A → C violates BCNF.

FD2: C → D

C^+ = {A, C, D}

Since B is missing, C is **not a superkey**.

Therefore:

C → D violates BCNF.

FD3: D → A

D^+ = {A, C, D}

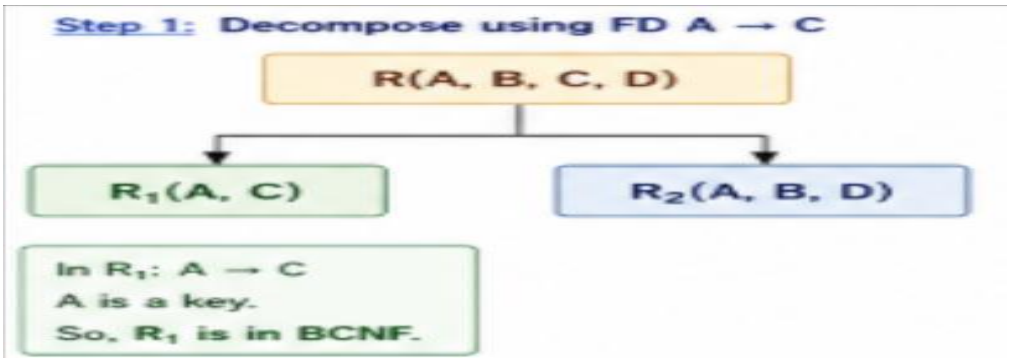
Since B is missing, D is **not a superkey**.

Therefore:

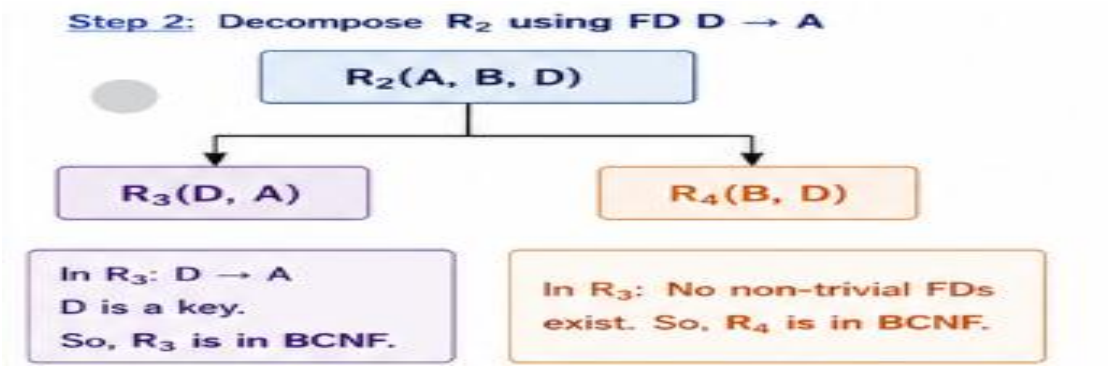
D → A violates BCNF.

3. BCNF DECOMPOSITION

Since the relation violates BCNF, we decompose it using the dependency:



4. **DECOMPOSE R2**



5. FINAL BCNF DECOMPOSITION

The original relation:

R(A, B, C, D)

is decomposed into:

R1(A, C)

R3(D, A)

R4(B, D)

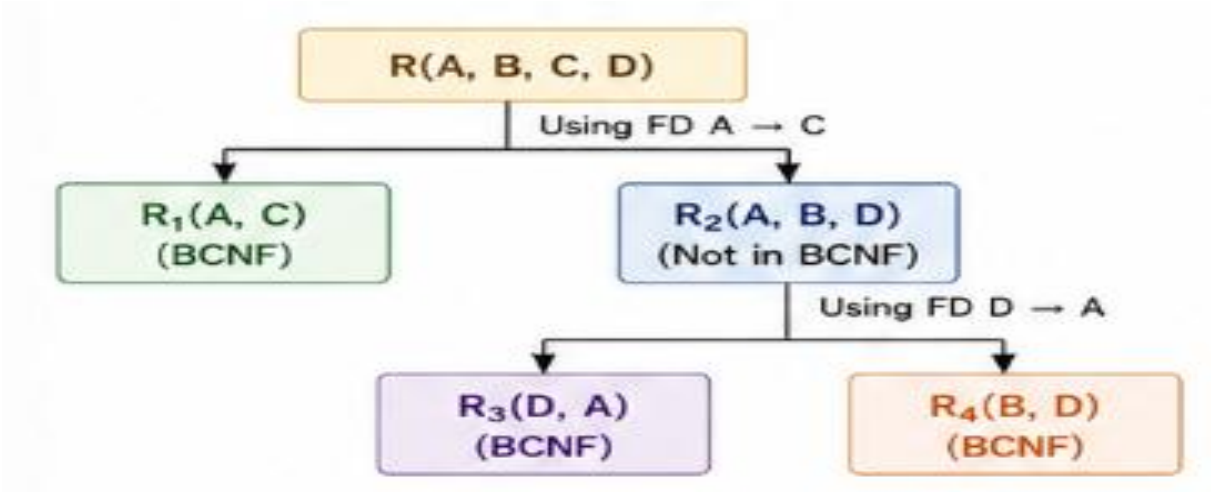
Final Relations

Relation	Attributes	Candidate Key	BCNF
R1	A, C	A	Yes
R3	D, A	D	Yes
R4	B, D	B,D	Yes

---

6. BCNF DECOMPOSITION DIAGRAM

Figure 5: BCNF Decomposition



RESULT

The relation **R(A, B, C, D)** is **not in BCNF** because the determinants **A, C, and D are not superkeys**.

After decomposition, the relations:

R1(A,C), R3(D,A), and R4(B,D) satisfy **BCNF**.

CONCLUSION

Thus, the given relation violates BCNF due to the functional dependencies **A → C, C → D, and D → A**. By decomposing the relation using these dependencies, three BCNF relations are obtained, reducing redundancy and improving database consistency.

QUESTION 6

Experimentally verify lossless-join decomposition for a given relation using the tableau (Chase) algorithm.

AIM

To verify whether a given decomposition of a relation is **lossless-join** using the **Tableau (Chase) Algorithm**.

1. GIVEN RELATION

Consider the relation:

R(A, B, C)

with the functional dependency:

B → C

The relation is decomposed into:

R_1(A, B)

R_2(B, C)

We need to verify whether this decomposition is **lossless**.

2. FUNCTIONAL DEPENDENCY

The given functional dependency is:

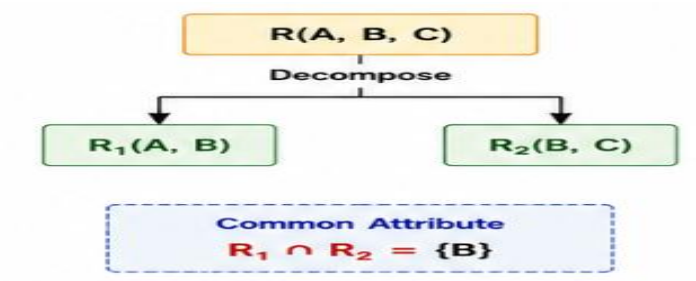
B → C

This means that the value of **B uniquely determines the value of C**.

3. DECOMPOSITION

The original relation is:

Row	A	B	C
$R_1(A,B)$	a	b	c_1
$R_2(B,C)$	a_2	b	c



The common attribute between the two relations is:
$R_1 \cap R_2 = \{B\}$

4. TABLEAU INITIALIZATION
For the Chase Algorithm, create one row for each decomposed relation.
Use distinguished symbols for attributes that are present in the corresponding decomposition.

Row	A	B	C
$R_1(A,B)$	a	b	c_1
$R_2(B,C)$	a_2	b	c

Here:
- a, b, c are distinguished symbols.
- c_1 and a_2 are non-distinguished symbols.

5. APPLY THE FUNCTIONAL DEPENDENCY
Given:
$B \rightarrow C$
Both rows have the same value for **B**:
R_1: B = b
R_2: B = b
Therefore, according to the functional dependency **$B \rightarrow C$**, their C-values must become equal.
The distinguished value of C in R_2 is
Therefore, replace c_1 in R_1 with c.
The tableau becomes:

Row	A	B	C
$R_1(A,B)$	**a**	**b**	**c**
$R_2(B,C)$	a_2	**b**	**c**

6. CHASE RESULT

Now observe the first row:

A	B	C
a	b	c

All the attributes in this row contain **distinguished symbols**.

Therefore, the chase algorithm succeeds.

Chase Condition

If a row becomes completely distinguished, then the decomposition is **lossless**.

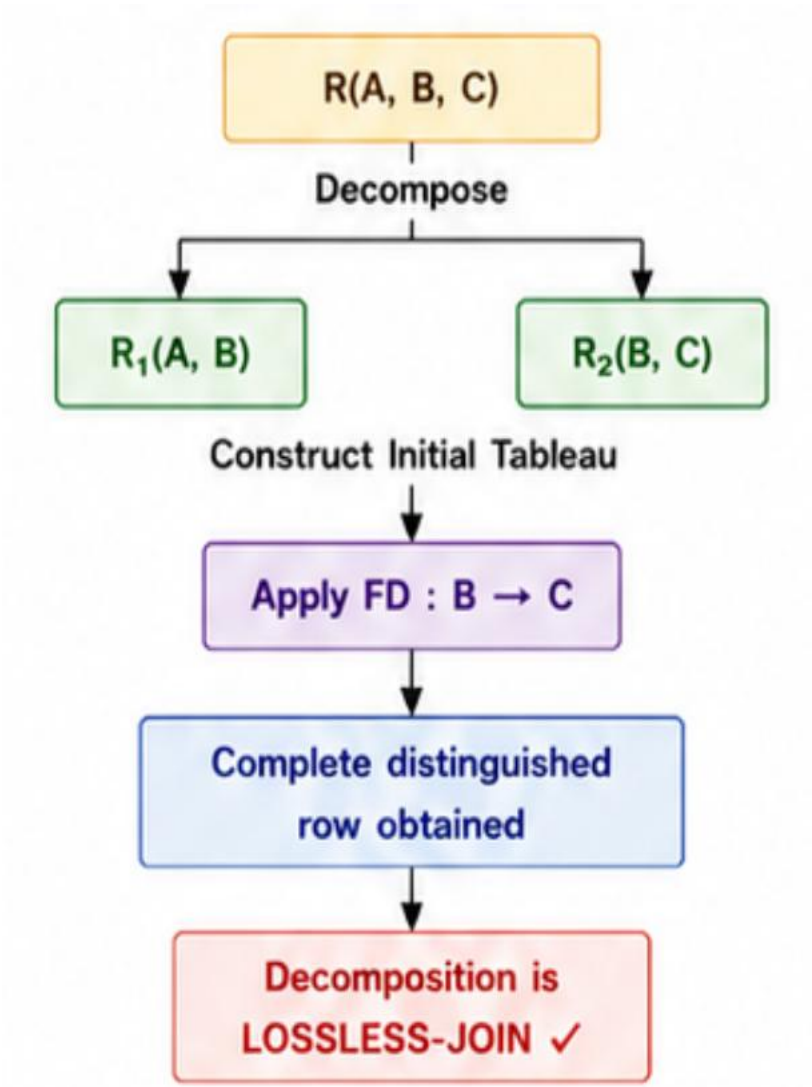
Therefore:

The decomposition R_1(A,B) and R_2(B,C) is lossless-join.

7. TABLEAU / CHASE DIAGRAM

Figure 6: Tableau (Chase) Algorithm for Lossless-Join Verification

Original Relation



8. VERIFICATION

Condition	Observation
Original relation	R(A,B,C)
Decomposition	R_1(A,B), R_2(B,C)
Common attribute	B
Functional dependency	B → C
Chase applied	B → C
Distinguished row obtained	Yes
Lossless decomposition	**Yes**

RESULT

Using the **Tableau (Chase) Algorithm**, the decomposition:

R(A,B,C) → R_1(A,B) and R_2(B,C)

was successfully verified as **lossless-join** because the chase produced a row containing all distinguished symbols.

CONCLUSION

Thus, the given decomposition is **lossless-join**. When the decomposed relations R_1 and R_2 are joined on their common attribute **B**, the original relation R can be reconstructed without generating spurious tuples.

QUESTION 7

Identify the multi-valued dependencies in **TEACHER(TID, Subject, Hobby)** and decompose it into **4NF**.

AIM

To identify the multivalued dependencies in the given TEACHER relation and decompose the relation into **Fourth Normal Form (4NF)** to eliminate redundancy caused by independent multivalued attributes.

GIVEN RELATION

TEACHER(TID, Subject, Hobby)

Where:

Attribute	Description
TID	Teacher ID
Subject	Subject taught by the teacher
Hobby	Hobby of the teacher

1. UNDERSTANDING THE RELATION

Consider the following example:

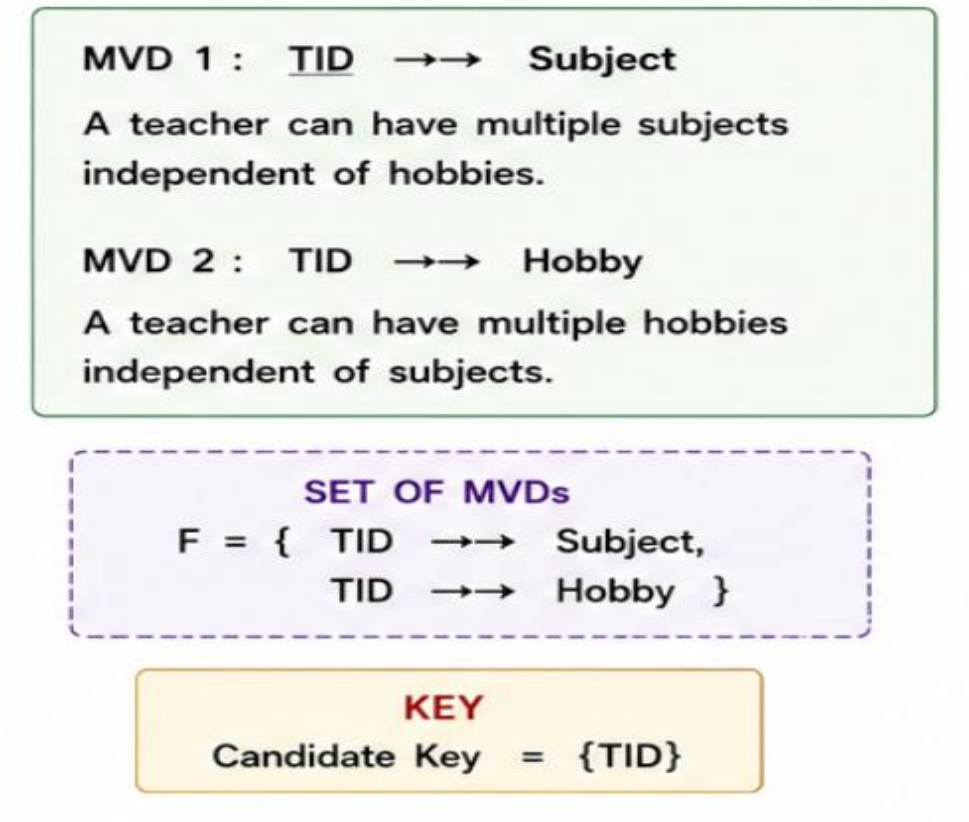
TID	Subject	Hobby

T01	DBMS	Reading
T01	DBMS	Music
T01	OS	Reading
T01	OS	Music
T02	CN	Sports
T02	DBMS	Reading

For teacher T01, the subjects and hobbies are independent of each other.
Subjects:
{DBMS, OS}
Hobbies:
{Reading, Music}
This produces all possible combinations between the subjects and hobbies.
This causes unnecessary repetition.

2. MULTIVALUED DEPENDENCIES

A multivalued dependency is represented by →→.
For the given relation, the following multivalued dependencies exist:



3. WHY THE MVDs EXIST

For a particular teacher, the subjects and hobbies are independent.
For example:

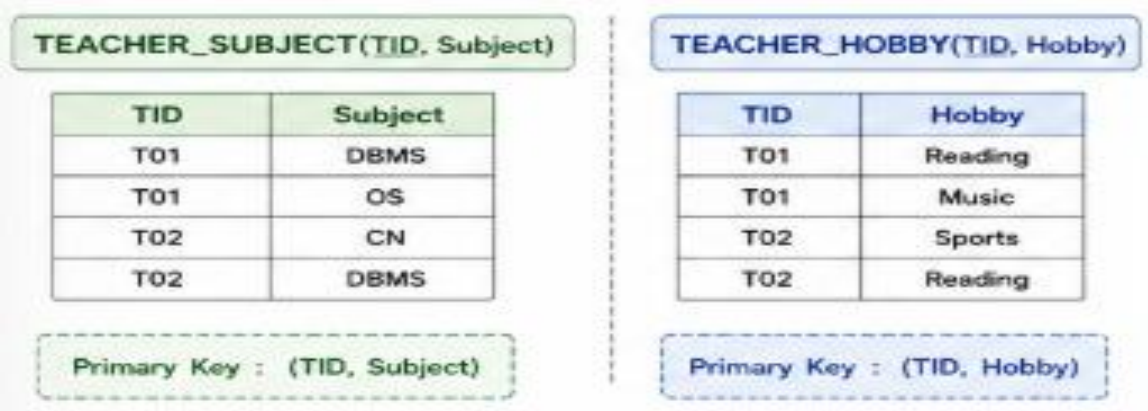
T01
Subjects = DBMS, OS
Hobbies = Reading, Music
Therefore, all combinations are generated:

TID	Subject	Hobby
T01	DBMS	Reading
T01	DBMS	Music
T01	OS	Reading
T01	OS	Music

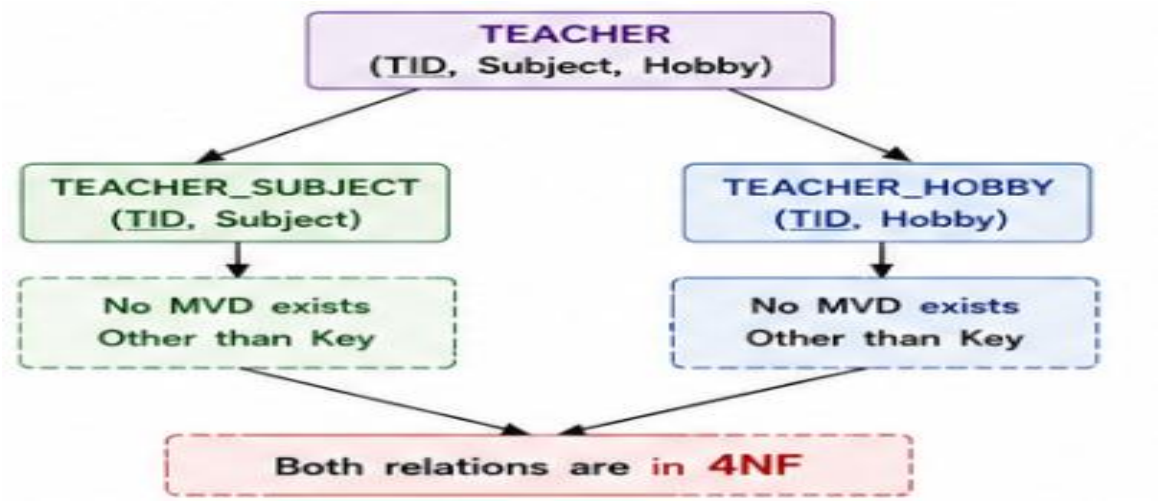
This creates **redundancy**.

4. CHECK FOR 4NF
A relation is in **Fourth Normal Form (4NF)** if, for every non-trivial multivalued dependency:
X →→ Y
X must be a **superkey**.
In the original relation:
TID →→ Subject
But TID alone does not determine one unique tuple of the entire relation, because a teacher can have multiple subjects and multiple hobbies.
Therefore, TID is not a superkey of the original relation.
Similarly:
TID →→ Hobby
also violates the 4NF condition.
Hence:

5. 4NF DECOMPOSITION
To remove the multivalued dependencies, decompose the relation into two relations.



6. 4NF DECOMPOSITION DIAGRAM
Figure 7: Decomposition of TEACHER Relation into 4NF



7. VERIFY 4NF

TEACHER_SUBJECT

TEACHER_SUBJECT(TID, Subject)

There is no independent multivalued attribute remaining.

Therefore, the relation satisfies 4NF.

TEACHER_HOBBY

TEACHER_HOBBY(TID, Hobby)

There is no independent multivalued attribute remaining.

Therefore, the relation satisfies 4NF.

8. COMPARISON

Original Relation	**4NF Relations**
TEACHER(TID, Subject, Hobby)	TEACHER_SUBJECT(TID, Subject)
	TEACHER_HOBBY(TID, Hobby)
Contains MVDs	MVD redundancy removed
Not in 4NF	Both relations satisfy 4NF
Data repetition	Reduced redundancy

RESULT

The multivalued dependencies **TID →→ Subject** and **TID →→ Hobby** were identified successfully. The original TEACHER relation was decomposed into:
TEACHER_SUBJECT(TID, Subject) And TEACHER_HOBBY(TID, Hobby)
Both resulting relations satisfy 4NF.

CONCLUSION

Thus, the TEACHER relation was successfully normalized to Fourth Normal Form by separating the independent multivalued attributes, Subject and Hobby. The decomposition reduces unnecessary repetition and improves data consistency.

QUESTION 8

Analyze the given relation for **Fifth Normal Form (5NF)** by identifying join dependencies and decompose the relation if required.

AIM

To identify join dependencies in a relation and determine whether the relation satisfies **Fifth Normal Form (5NF)**, also called **Project-Join Normal Form (PJ/NF)**.

GIVEN RELATION

Consider the relation:

SUPPLIER_PART_PROJECT(Supplier, Part, Project)

Where:

Attribute	Description
Supplier	Supplier providing the part
Part	Part supplied
Project	Project for which the part is supplied

1. SAMPLE RELATION

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S2	P1	J1
S2	P2	J1

The relation contains information about the association between **Supplier, Part, and Project**.

2. JOIN DEPENDENCY

A **Join Dependency (JD)** exists when a relation can be reconstructed by joining two or more of its projections.

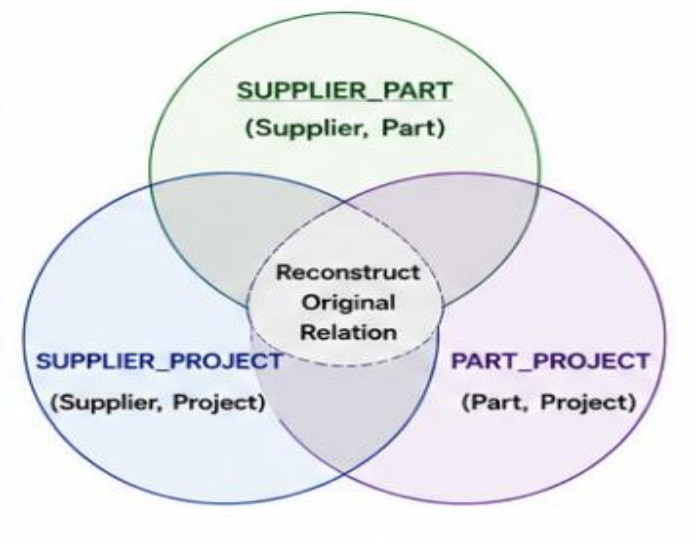
For the given relation, assume the following join dependency:

(Supplier, Part), (Supplier, Project), (Part, Project)

This means that the original three-attribute relation can be represented using three smaller relations.

Therefore:

$$R = \pi_{Supplier,Part}(R) \bowtie \pi_{Supplier,Project}(R) \bowtie \pi_{Part,Project}(R)$$



3. CHECK FOR 5NF

A relation is in **5NF** if every non-trivial join dependency is implied by the candidate keys of the relation.

In the given relation, the three attributes participate in a join dependency:

(Supplier, Part), (Supplier, Project), (Part, Project)

This join dependency is not represented by a single candidate key.

Therefore, the relation requires decomposition to eliminate the non-trivial join dependency.

Hence:

The original relation is not in 5NF.

4. 5NF DECOMPOSITION

The relation is decomposed into three relations.

Relation 1: SUPPLIER_PART

SUPPLIER_PART(Supplier, Part)

Supplier	Part
S1	P1
S1	P2
S2	P1
S2	P2

Relation 2: SUPPLIER_PROJECT

SUPPLIER_PROJECT(Supplier, Project)

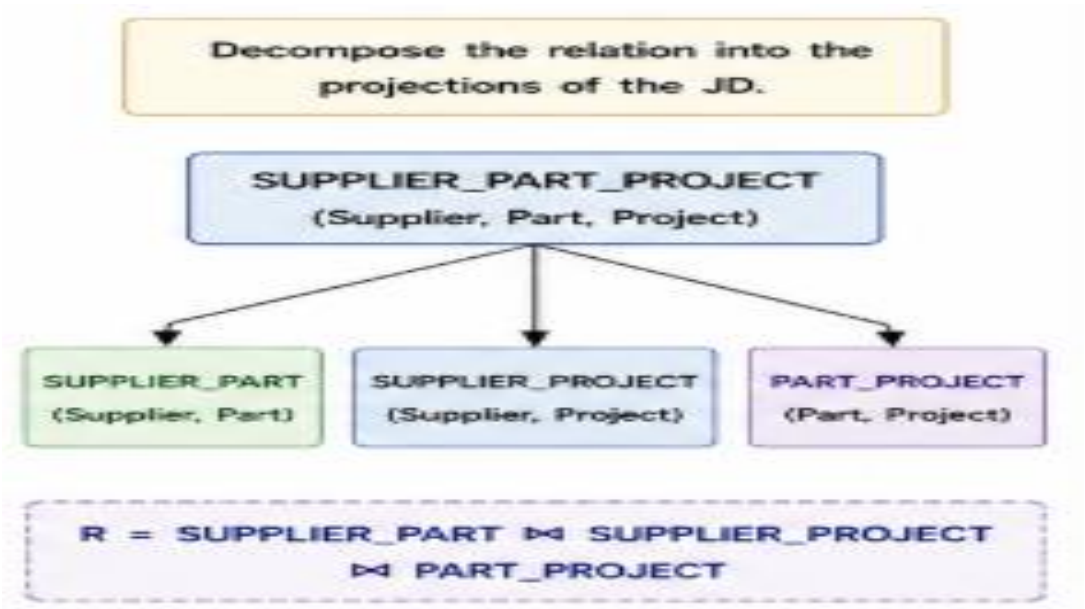
Supplier	Project
S1	J1
S1	J2
S2	J1

Relation 3: PART_PROJECT
PART_PROJECT(Part, Project)

Part	Project
P1	J1
P1	J2
P2	J1

5. 5NF DECOMPOSITION DIAGRAM

Figure 8: 5NF Decomposition



Supplier, Part Supplier, Project Part, Project
The original relation can be reconstructed by joining the three decomposed relations.

6. VERIFY 5NF

After decomposition:

SUPPLIER_PART
Contains only Supplier-Part relationships.
SUPPLIER_PROJECT
Contains only Supplier-Project relationships.
PART_PROJECT
Contains only Part-Project relationships.

No further non-trivial join dependency needs to be decomposed.
Therefore, the resulting relations satisfy **5NF**.

7. COMPARISON

Before Decomposition	After Decomposition
Supplier, Part, Project in one relation	Three separate relations
Contains non-trivial JD	JD is represented by projections
Redundancy possible	Redundancy reduced
Not in 5NF	Relations satisfy 5NF

RESULT

The join dependency in the **SUPPLIER_PART_PROJECT** relation was identified successfully. The relation was decomposed into:

1. **SUPPLIER_PART(Supplier, Part)**
2. **SUPPLIER_PROJECT(Supplier, Project)**
3. **PART_PROJECT(Part, Project)**

The decomposition eliminates the non-trivial join dependency and results in relations satisfying **5NF**.

CONCLUSION

Thus, the relation was successfully analyzed and decomposed into Fifth Normal Form. The decomposition reduces redundancy and ensures that the original relation can be reconstructed through appropriate joins without storing unnecessary repeated combinations.

QUESTION 9

Identify insertion, deletion, and update anomalies in a relation and normalize the relation up to 3NF.

AIM

To identify the **insertion, deletion, and update anomalies** in an unnormalized relation and eliminate these anomalies by decomposing the relation into **Third Normal**

Form (3NF).

GIVEN RELATION

Consider the following relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor)

SID	SName	CID	CName	Instructor
S101	Jaya	C01	DBMS	Kumar
S101	Jaya	C02	Operating Systems	Ravi
S102	Priya	C01	DBMS	Kumar

S103	Arun	C03	Computer Networks	Meena

Functional Dependencies

1. **SID → SName**
2. **CID → CName**
3. **CID → Instructor**

The candidate key is:

(SID, CID)

1. INSERTION ANOMALY

An **insertion anomaly** occurs when new information cannot be inserted without providing unrelated information.

Example

Suppose a new course is introduced:

CID = C04

CName = Artificial Intelligence

Instructor = Meena

But currently no student has enrolled in this course.

In the original relation, it is difficult to store the course information because SID and SName are required as part of the record.

Therefore, the course information cannot be stored independently.

This is an insertion anomaly**.**

2. DELETION ANOMALY

A **deletion anomaly** occurs when deleting one record unintentionally removes other important information.

Example

Suppose student S103 is the only student enrolled in:

C03 – Computer Networks

If S103's record is deleted, the information about:

C03 – Computer Networks – Meena

is also lost.

Therefore, deleting a student can unintentionally delete course information.

This is a deletion anomaly.

3. UPDATE ANOMALY

An **update anomaly** occurs when the same information is stored in multiple rows and must be updated in several places.

Example

The instructor for **C01 – DBMS** is currently **Kumar**.

C01 appears for multiple students:

SID	SName	CID	CName	Instructor

S101	Jaya	C01	DBMS	Kumar
S102	Priya	C01	DBMS	Kumar

If the instructor changes from Kumar to Ravi, every occurrence must be updated.
If one row is not updated, inconsistent data will occur.
This is an update anomaly.

4. NORMALIZATION TO 3NF

To remove these anomalies, the relation is decomposed based on the functional dependencies.

Relation 1: STUDENT

STUDENT(SID, SName)

SID	SName
S101	Jaya
S102	Priya
S103	Arun

Primary Key: SID

Functional dependency:

SID → SName

Relation 2: COURSE

COURSE(CID, CName, Instructor)

CID	CName	Instructor
C01	DBMS	Kumar
C02	Operating Systems	Ravi
C03	Computer Networks	Meena

Primary Key: CID

Functional dependencies:

CID → CName, Instructor

Relation 3: ENROLLMENT

ENROLLMENT(SID, CID)

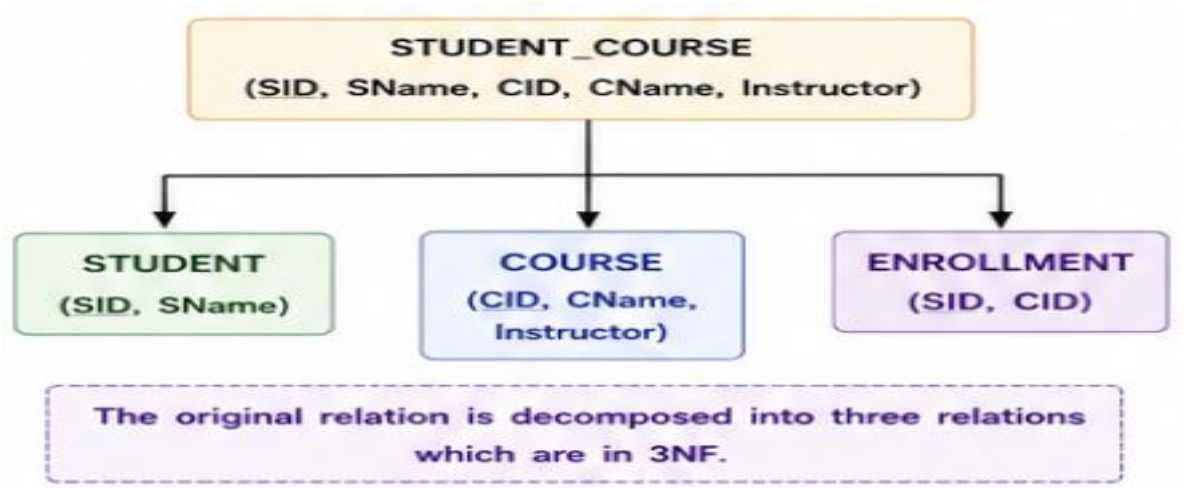
SID	CID
S101	C01
S101	C02
S102	C01
S103	C03

Primary Key: (SID, CID)

This relation connects students with the courses they take.

5. 3NF DECOMPOSITION DIAGRAM

Figure 9: Normalization of STUDENT_COURSE to 3NF



6. HOW ANOMALIES ARE REMOVED

Anomaly	Solution
Insertion anomaly	New students and courses can be added independently
Deletion anomaly	Deleting a student does not delete course information
Update anomaly	Course/instructor information is stored only once
Data redundancy	Repeated information is reduced
Data consistency	Improved through separate relations

7. 3NF VERIFICATION

STUDENT

SID → SName

SID is the candidate key.

Therefore, STUDENT is in 3NF.

COURSE

CID → CName, Instructor

CID is the candidate key.

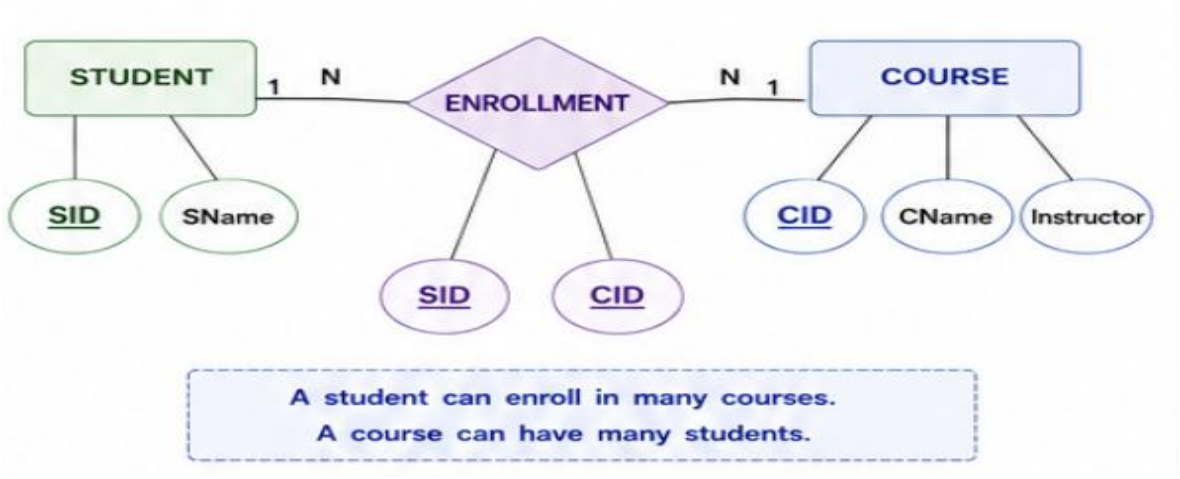
Therefore, COURSE is in 3NF.

ENROLLMENT

(SID, CID) is the candidate key.

There are no non-key attributes.

Therefore, ENROLLMENT is in 3NF.



RESULT

The insertion, deletion, and update anomalies in the STUDENT_COURSE relation were successfully identified and eliminated by decomposing the relation into:

STUDENT(SID, SName)
COURSE(CID, CName, Instructor)
ENROLLMENT(SID, CID)

All three relations satisfy Third Normal Form (3NF).

CONCLUSION

Thus, normalization to 3NF successfully removes insertion, deletion, and update anomalies. The decomposition reduces redundancy, improves data consistency, and allows student, course, and enrollment information to be maintained independently.

QUESTION 10

Given the relation **R(A, B, C, D, E)** with functional dependencies **A → B, B → C, CD → E**, compute the attribute closure and determine the candidate key.

AIM

To calculate the attribute closure of the given attributes and determine the candidate key of the relation using functional dependencies.

GIVEN RELATION

R(A, B, C, D, E)

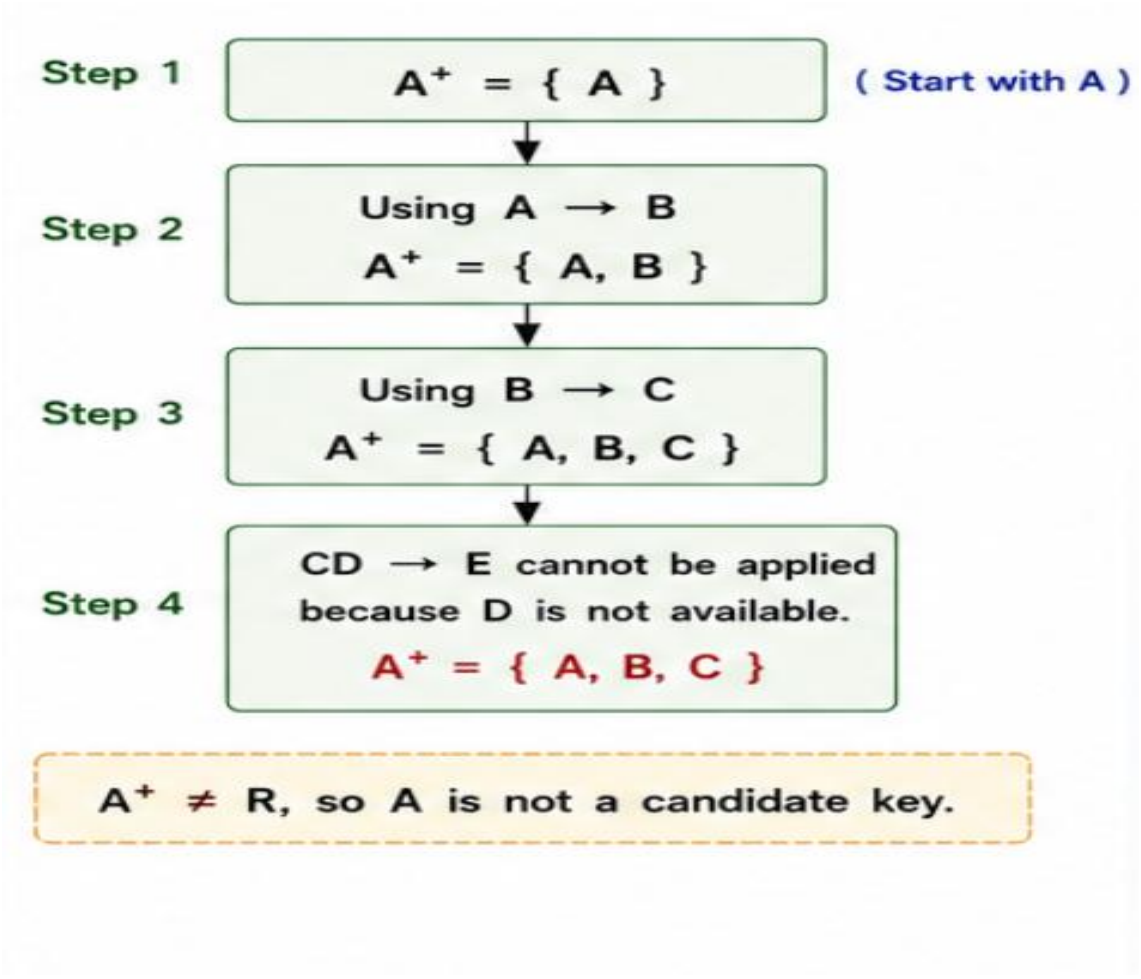
Functional Dependencies

1. **A → B**
2. **B → C**
3. **CD → E**

Therefore:

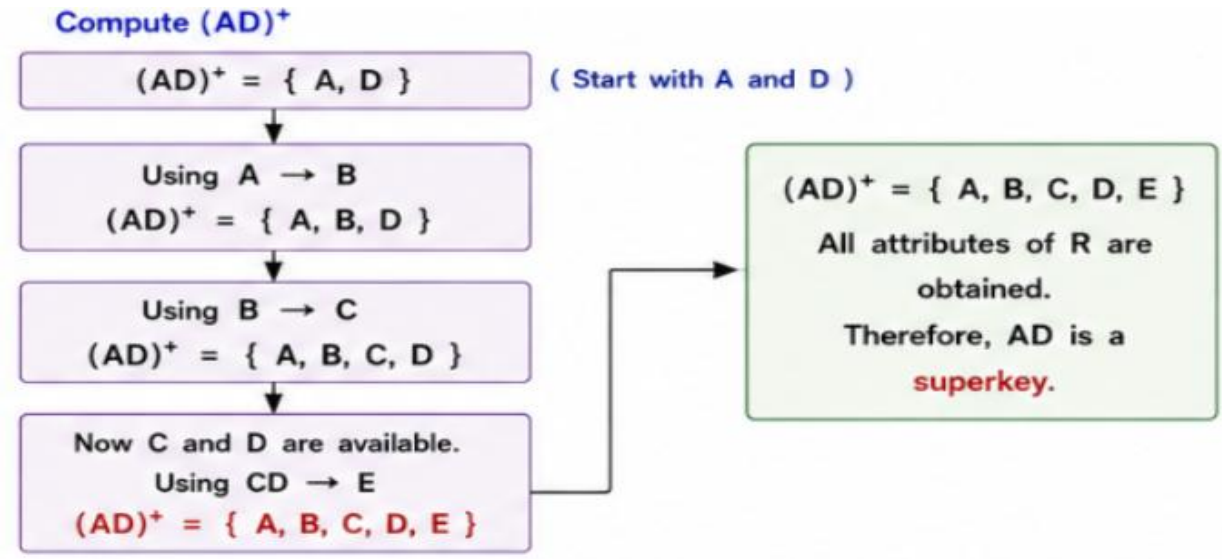
F = { A → B, B → C, CD → E }

1. ATTRIBUTE CLOSURE OF A



2. FIND THE CANDIDATE KEY

Notice that **D does not appear on the right-hand side of any functional dependency**.
Therefore, **D must be included in every candidate key**.
Consider **AD**.



3. MINIMALITY CHECK

Check whether either attribute can be removed.

A alone

A^+ = {A, B, C}

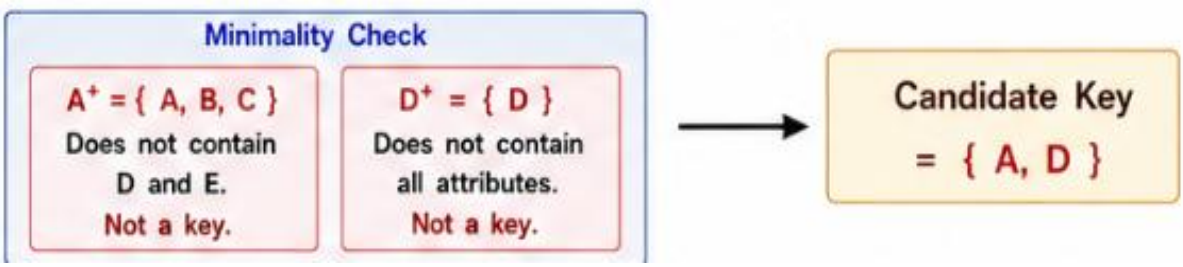
It does not contain D and E.
Therefore, A alone is not a key.

D alone

There is no dependency starting from D.
Therefore:

D^+ = {D}

It does not contain all attributes.
Therefore, D alone is not a key.
Hence, both A and D are necessary.

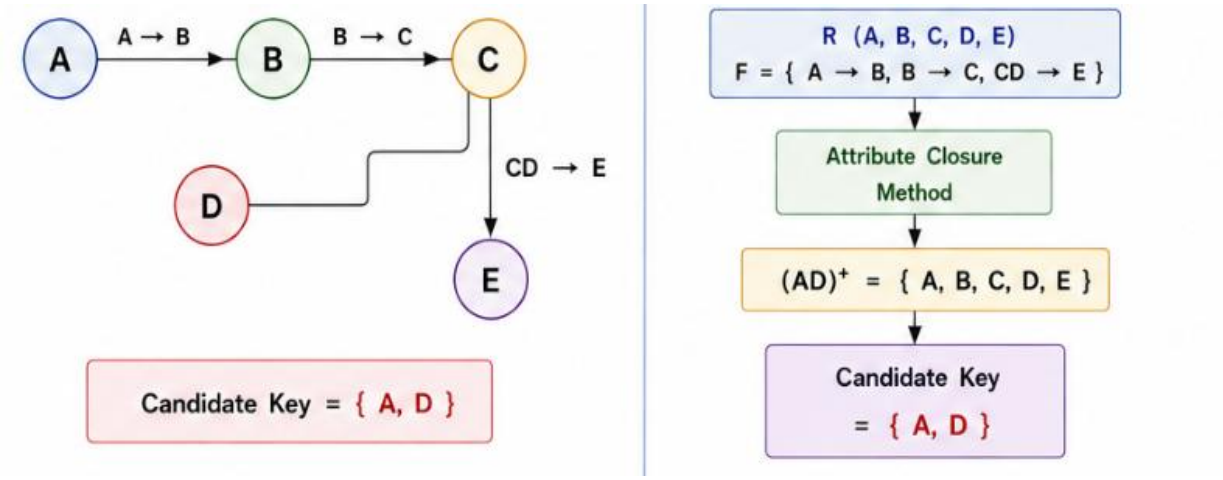


4. ATTRIBUTE CLOSURE TABLE

Attribute Set	Closure	Candidate Key?
A	{A, B, C}	No
D	{D}	No
AD	{A, B, C, D, E}	**Yes**

5. CLOSURE DIAGRAM

Figure 10: Attribute Closure and Candidate Key



6. VERIFICATION

For AD:
AD → B because A → B
AD → C because A → B and B → C
AD → E because AD gives C and D, and CD → E
Therefore:
$AD^+ = \{A, B, C, D, E\}$
Hence, AD determines every attribute in the relation.

RESULT

The attribute closure of A is:
$A^+ = \{A, B, C\}$
The attribute closure of AD is:
$(AD)^+ = \{A, B, C, D, E\}$
Therefore, the candidate key of the relation is:

CONCLUSION

Thus, the attribute closure method was successfully used to determine the candidate key. Since D does not occur on the right-hand side of any functional dependency, it must be included in the candidate key. Combining D with A gives all attributes of the relation, making {A, D} the minimal candidate key.